

Backstop: Recovery-Safe GPU Sharing Between Neural Receivers and AI Inference in AI-RAN

ANONYMOUS AUTHOR(S)

AI-RAN servers run the uplink physical layer and AI inference on the same GPUs. Neural receivers decode transport blocks (TBs) that the conventional receiver fails, but their value and their cost on a shared GPU are not established. We compare public neural receivers with the cuPHY receiver at the same LDPC iteration count and find that the real-time model helps only when the channels of paired users are correlated, while a larger model decodes 15-67% of the failed TBs on every channel we test. The larger model takes 5.3 ms per slot, twice the uplink period. This paper presents Backstop, which runs the larger model only for failed TBs under a later deadline and grants AI inference in short pieces that keep both the layer-1 deadline and the recovery deadline. On four A100 GPUs with 16 cells, Backstop recovers 63-66% of the failed TBs on the low-correlation channel and 38% with mixed channels. Under a load that changes every two seconds, Backstop serves 27.4k tokens/s while keeping 98.6% of the recoveries, $5.0\times$ a fixed GPU share and $1.17\times$ a share that follows the load, at the same recovery level.

CCS Concepts: • **Computer systems organization** → **Real-time systems**; • **Networks** → *Network resources allocation*.

Additional Key Words and Phrases: AI-RAN, GPU scheduling, neural receiver, MPS, admission control

ACM Reference Format:

Anonymous Author(s). 2027. Backstop: Recovery-Safe GPU Sharing Between Neural Receivers and AI Inference in AI-RAN. In *Proceedings of ACM SIGMETRICS 2027 (SIGMETRICS '27)*. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Radio access networks (RANs) increasingly run baseband processing on GPUs, and AI-RAN platforms run radio physical-layer (PHY) processing and AI inference on the same GPUs [3, 13, 18, 21]. To use the GPU time that radio processing leaves idle, these platforms add AI services, such as large language model (LLM) inference, next to deadline-critical PHY tasks [12, 20]. Neural receivers are a PHY workload that these platforms now run on the GPU [13, 23]. A neural receiver replaces channel estimation, equalization, and demapping with a learned model and decodes transport blocks (TBs) that a conventional receiver fails [6, 10, 23]. Each TB that it decodes avoids a retransmission and frees an uplink slot.

The challenge stems from two facts that we measure in this paper. First, the neural receiver that helps is expensive. With the same number of LDPC iterations in both receivers, the real-time model of the public NVlabs receivers [7, 23] decodes more TBs than the cuPHY receiver only when the channels of the paired users are correlated. The larger model decodes more TBs on every channel we test, and it takes 5.3 ms per slot against an uplink period of 2.5 ms. Thus, it cannot run for every slot. Second, the larger model shares the GPU with the conventional receivers of all cells and with AI inference. NVIDIA Multi-Process Service (MPS) caps the GPU threads of a process, but the cap is fixed when the process starts [14]. A cap that protects the radio work at full load leaves the GPU idle when the load drops, and a cap that fits a light load delays the radio work at full load.

Existing systems cover one side of this problem. As Table 1 summarizes, vRAN and AI-RAN resource managers [3, 8, 12, 18–21] share GPUs between radio processing and other workloads by a GPU share or by reserved capacity, and their radio work is LDPC decoding or the conventional PHY. GPU scheduling systems [4, 5, 9, 22] prioritize or preempt GPU kernels and contexts without radio deadlines. Neural receiver studies and PHY platforms [6, 10, 15, 17, 23] design the receiver or switch between PHY models, and they do not share the GPU with an AI workload. However, none

Table 1. What the closest public designs run on the shared GPU.

System	Radio work on the GPU	How AI shares the GPU
YinYangRAN, CloudRIC, Concordia	LDPC decoding or queued PHY jobs	GPU share or reserved capacity, changed at intervals of one second or more
DARIS, REEF, XSched, nvtaskset	None; generic GPU tasks	Kernel, context, or partition priority without radio deadlines
ARCHES, OCUDU, neural receivers	Neural channel estimator or neural receiver	No AI workload on the same GPU
Backstop	Conventional receiver for every TB and a neural receiver for failed TBs	AI pieces granted against the layer-1 deadline and the recovery deadline

of them runs a neural receiver next to AI inference on one GPU while keeping the deadlines of both radio paths.

In this paper, we present Backstop, a GPU sharing scheme for AI-RAN servers that uses a neural receiver as a second receiver for the TBs the conventional receiver fails. Specifically, Backstop introduces 1) a recovery path that sends a failed TB to the neural receiver only when the number of failed code blocks predicts a successful decode, 2) two deadlines per TB, the layer-1 deadline for the conventional receiver and a later recovery deadline before the retransmission is scheduled, and 3) AI admission in bounded pieces, where the controller allows a piece only if every running and waiting radio job still meets its deadline. Our evaluation on four A100 GPUs with 16-32 cells, the cuPHY receiver, the public NVlabs neural receivers, and Qwen2.5 prefill requests shows that Backstop recovers 63-66% of the TBs the conventional receiver fails on the low-correlation channel and 38% with mixed channels. With mixed channels and a load that alternates between full and half every two seconds, Backstop serves 27.4k tokens/s within the 200 ms limit and keeps 98.6% of the recoveries. A fixed 10% GPU share keeps the same recoveries at 5.5k tokens/s, and a share that switches between 10% and 50% with the load, with no switching cost, reaches 23.5k tokens/s. When the neural receiver load is light, a fixed share serves more AI than Backstop, and we report those settings as well.

2 Background and Motivation

2.1 AI-RAN on shared GPUs

An AI-RAN server decodes the uplink of many cells on GPUs and serves AI requests with the GPU time that is left [3, 13].

Uplink Timeline. With the TDD pattern DDDSU at 30 kHz subcarrier spacing, every cell has one uplink slot per 2.5 ms. The received samples of a slot are ready 0.5 ms after the slot starts, and the layer-1 result is due 4.5 ms after the slot starts, which leaves 4.0 ms for decoding. A TB that fails is retransmitted three uplink slots later, 7.5 ms after the original slot: the grant goes out in the special slot before that uplink slot with a scheduling offset of one slot, which the PUSCH preparation time of 12 symbols allows [2].

GPU Sharing. MPS runs the processes of one GPU concurrently and caps the active threads of each process [14]. YinYangRAN changes the share between LDPC decoding and other workloads with a learned predictor at intervals of at least one second [18], and CloudRIC pools GPUs for LDPC decoding across distributed units [19]. The share of an MPS client cannot change after the client starts. Therefore, a system that follows the load must restart the client or keep one loaded client per share.

Table 2. TBs decoded at 20 LDPC iterations in both receivers. The percentage is the share of the TBs cuPHY fails that the neural receiver decodes; the time is the GPU time of the model alone.

Channel of the two users	TBs	cuPHY	Real-time model (1.35 ms)	Larger model (4.29 ms)
Low correlation, 2-3 dB	600	515	484 (2%)	572 (67%)
Urban micro, 1-6 dB	600	425	420 (2%)	451 (15%)
Medium correlation, 8-18 dB	600	321	372 (21%)	not measured
High correlation, 10-16 dB	1,400	946	1,099 (38%)	1,159 (51%)

2.2 When does a neural receiver help?

We compare the public NVlabs neural receivers [6, 7, 23] with the cuPHY PUSCH receiver [13] on the same received slots. Two users transmit on the same 273 PRBs with 16QAM, and the base station has four antennas. The slots come from the NVlabs link simulation [11]; cuPHY decodes them with MMSE equalization and time interpolation.

Same LDPC Effort. A receiver comparison is valid only when both receivers run the same number of LDPC iterations. cuPHY uses 10 iterations for the TB sizes in our setup, and the evaluation code of the neural receivers uses 20. With 10 against 20 iterations, the real-time model appears to decode 25-35% of the TBs that cuPHY fails on the low-correlation channel. With 20 iterations on both sides, the figure is 2%. Raising cuPHY from 10 to 20 iterations alone recovers 152 of 158 failed single-user TBs, at 15-19% more receiver time for TBs of 13-20 code blocks. Therefore, every result in this paper uses 20 iterations in both receivers. Table 2 shows the comparison.

The Model That Helps Is Too Slow for Every Slot. The real-time model decodes more TBs than cuPHY only when the channels of the two users are correlated. The larger model decodes more TBs on all three channels and loses none of the TBs that cuPHY decodes on two of them. It takes 5.3 ms per slot alone and 6.3 ms next to the cell workload, against an uplink period of 2.5 ms; cuPHY takes 0.8-0.9 ms. When we send every slot of four cells to the larger model, 61% of the slots find no free GPU in time and 89.4% of the TBs are decoded. When we send only the TBs that cuPHY fails, the neural receiver runs for 25% of the slots, the decode result is the same offline, and 94.5-95.1% of the TBs are decoded in the timed system.

Failed Code Blocks Predict the Outcome. The conventional receiver reports how many code blocks of a TB failed. On the urban micro channel, 87% of the failed TBs have every code block failed and the neural receiver decodes 2% of them. It decodes 93% of the TBs in slots with at most nine failed code blocks. On the low-correlation channel, it decodes 97% of the failed TBs in slots with one to three failed code blocks and 30% in slots with ten or more.

This motivates Backstop's design: run the larger model only for failed TBs that it is likely to decode, give that run a deadline later than the layer-1 deadline, and admit AI only where both deadlines hold.

3 Backstop Design

3.1 Overall architecture

Backstop runs four kinds of processes on each server (Figure 1). A conventional receiver process per cell decodes the TBs of its cell with cuPHY every 2.5 ms. A neural receiver process per GPU decodes one TB at a time. An AI worker per GPU runs AI inference in pieces. One controller assigns failed TBs to neural receivers, grants AI pieces, and places AI requests on GPUs. All processes read one shared state in memory and one clock.

The steps for one slot are as follows. ❶ The samples of a slot arrive in the slot ring of the cell, which stays in the GPU memory of its conventional receiver, and cuPHY decodes them. ❷ The

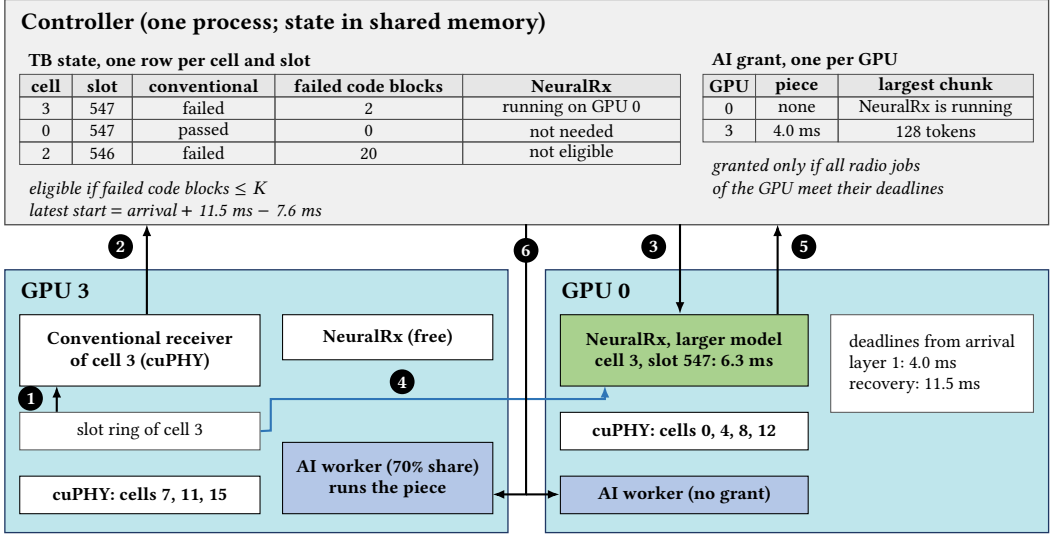


Fig. 1. Overall architecture of Backstop with the values of one recovery in a measured run (two of the four GPUs are shown). Numbers mark the steps of the text.

receiver writes the CRC result and the number of failed code blocks to the TB state. ③ The controller assigns an eligible failed TB to a free neural receiver before the latest start time of the TB. ④ Every neural receiver process maps the slot rings of all cells. Thus, the neural receiver copies the slot from the GPU of the cell and decodes it on its own GPU. ⑤ A result that arrives before the recovery deadline cancels the retransmission. ⑥ In every round the controller writes one AI grant per GPU, and the AI worker runs only inside the granted piece.

3.2 Recovery path

Two Deadlines. Every TB has a layer-1 deadline of 4.0 ms after its samples arrive, which the conventional receiver must meet. A TB that fails has a *recovery deadline*: if the neural receiver decodes it before this deadline, the retransmission is cancelled. A recovery deadline of 6.5 ms keeps the retransmission slot of the conventional path. The larger model does not fit into 6.5 ms, and Backstop uses 11.5 ms, which delays the retransmission of a TB that is not recovered by two uplink slots (5 ms).

Which TBs Are Recovered. A failed TB becomes a candidate only if its slot has at most K failed code blocks. The controller computes the latest start time of a candidate as its arrival plus the recovery deadline minus the time bound of the neural receiver. It assigns candidates in order of latest start time to a free neural receiver, preferring a GPU with no other neural receiver running and no AI piece running. A candidate that passes its latest start time is dropped and its TB is retransmitted. A neural receiver result counts only if the CRC passes, the payload is correct, and the run ends before the recovery deadline.

3.3 AI admission in bounded pieces

AI Units. The AI worker splits the prefill of an LLM request into units of one layer over a chunk of 128, 512, or 1,024 tokens, each compiled as one GPU graph with a measured time bound. A larger chunk is faster per token and holds the GPU longer once it starts.

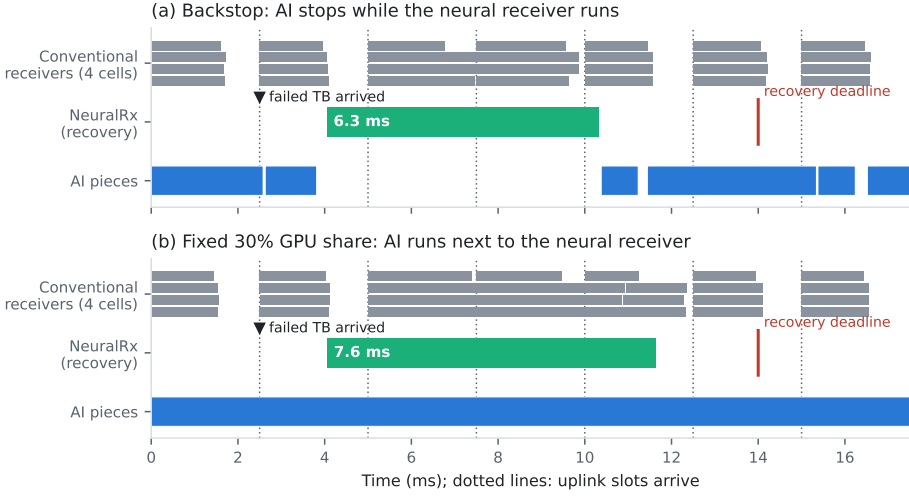


Fig. 2. Measured timeline of GPU 0 around one recovery (the run of Figure 1, same seed for both policies). Bars show when each job runs.

Grants. The AI worker runs under an MPS share of 70% and submits units only inside a piece that the controller grants. For each GPU the controller picks the largest chunk size c that meets all of the following and grants a piece of at most 4 ms:

- *Running neural receiver.* For every neural receiver running on this GPU, its start time plus $B(c)$ is before its recovery deadline, where $B(c)$ is the time bound of the neural receiver next to AI units of size c .
- *Conventional receivers.* If size c is not allowed next to the conventional receivers, no cell of this GPU is decoding and the piece ends before the next slot arrives.
- *Neural receiver about to start.* If a TB of this GPU may still become a candidate, the piece ends before the latest start time of that TB under $B(c)$.
- *Waiting candidates.* No candidate is waiting for a free neural receiver.

If no size meets the conditions, the GPU gets no AI in this round. The worker keeps the chunk size of a request stable by choosing the largest size that was allowed for most of the last 20 ms.

Figure 2 shows the effect on a measured run. Under Backstop, the AI worker stops before the neural receiver starts, the neural receiver finishes in 6.3 ms, and the AI worker resumes after it. Under a fixed 30% share, AI runs next to the neural receiver, which takes 7.6 ms, and next to the conventional receivers, which finish later.

AI Requests. The controller predicts when a new request would finish from the tokens that wait on each GPU and the recent prefill rate of that GPU. It places the request on the GPU that finishes it first and rejects it if no GPU finishes it within the limit. Every baseline uses the same placement and rejection.

3.4 Time bounds and implementation

The bounds come from measurements of the 99.9th percentile on the evaluation hardware with all cells running. With 16 cells, the larger neural receiver takes 6.3 ms at the median and the bound is 7.6 ms alone and 9.8-10.3 ms next to AI units. The neural receiver path is the public TensorRT export of the NVlabs model followed by rate recovery, LDPC decoding, and CRC from cuPHY.

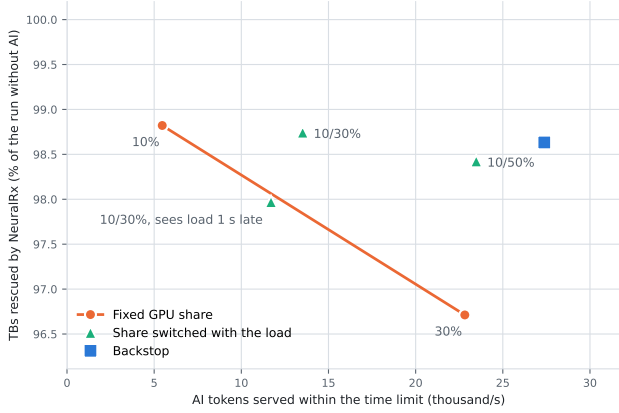


Fig. 3. AI served and recoveries kept when the radio load alternates between full and half every 2 s (16 cells, mixed channels, five seeds). Labels give the MPS shares.

We replace the least-squares channel estimation call with one multiplication of the pilot samples and call the cuPHY bit-level functions on buffers allocated once, which shortens the path of the real-time model from 3.08 to 2.36 ms with the same decoded TBs. The run-time path of the real-time model agrees with the NVlabs evaluation on 794 of 800 TBs. Backstop is 2.9k lines of Python over cuPHY, TensorRT, and PyTorch.

4 Evaluation

4.1 Evaluation setup

Hardware. One node with four NVIDIA A100 80 GB GPUs under MPS and 64 CPU cores.

Scope. All measurements are end-to-end runs on the GPUs: cuPHY decodes every TB, the neural receiver decodes real received slots, and the AI worker runs real prefill. The slots are generated in advance; there is no radio unit and no layer-2 stack.

Workloads. Sixteen cells run on four GPUs, four per GPU, unless stated. One cell per GPU carries two users on the same 273 PRBs (16QAM, 20 code blocks per slot); the other cells carry one user with two layers. The two-user slots come from three channels of the NVlabs link simulation: low correlation at 2-3 dB, high correlation at 13-16 dB, and urban micro [1] at 4-6 dB. Both receivers run 20 LDPC iterations, the neural receiver is the larger NVlabs model, $K = 9$ with mixed channels, and the recovery deadline is 11.5 ms. AI requests are Qwen2.5-1.5B [16] prefill requests of up to 1,024 tokens with a 200 ms limit, offered at 32 requests/s per GPU. A request that finishes after the limit does not count. Runs last 10-20 s; values are means over seeds.

Baselines. *Fixed share* gives the AI worker a fixed MPS share and no grants. *Share that follows the load* keeps one AI worker per share loaded on every GPU and makes the low-share worker active while the GPU is at full load and the high-share worker otherwise. It has no switching cost, the waiting requests pass to the active worker, and we run it with the load known at once and known one second late, the minimum reconfiguration interval of YinYangRAN [18]. All policies use the same recovery path, the same AI requests, and the same placement and rejection of AI requests. We report the recoveries of a run as a share of the recoveries of the run without AI on the same seed.

Table 3. Changing load, 16 cells, mixed channels, five seeds (range over seeds in parentheses).

Policy	AI tokens/s within 200 ms	Recoveries kept
Backstop	27.4k (26.0-28.5k)	98.6% (97.6-99.3%)
Fixed 10%	5.5k (5.3-5.6k)	98.8% (97.4-99.8%)
Fixed 30%	22.8k (21.9-23.5k)	96.7% (95.5-98.6%)
Share follows load, 10/30%	13.5k (12.6-14.6k)	98.7% (97.7-99.6%)
Share follows load, 10/50%	23.5k (22.9-24.6k)	98.4% (97.3-99.3%)
Share follows load, 10/30%, 1 s late	11.7k (11.5-11.8k)	98.0% (96.6-99.2%)

Table 4. Steady full load, 16 cells. Mixed channels with five seeds; one channel with three seeds.

Policy	Mixed channels		Low correlation, $K = 20$	
	Tokens/s	Recoveries kept	Tokens/s	Recoveries kept
Backstop	22.7k	97.9%	12.6k	98.2%
Fixed 10%	5.0k	98.3%	4.4k	97.5%
Fixed 20%	10.8k	97.3%	9.0k	96.4%
Fixed 30%	20.9k	95.1%	15.9k	93.8%
Fixed 50%	36.3k	91.9%	25.9k	87.0%

4.2 AI served under a changing load

The radio load alternates every two seconds between full load and each cell active with probability 0.5. Figure 3 and Table 3 show AI served against recoveries kept. Backstop serves 27.4k tokens/s and keeps 98.6% of the recoveries. The fixed 10% share keeps 98.8% and serves 5.5k tokens/s, $5.0\times$ less. The fixed 30% share serves 22.8k tokens/s and loses 3.3% of the recoveries, $2.4\times$ more than Backstop. The share that follows the load serves 13.5-23.5k tokens/s at the same recovery level, $1.17\text{-}2.0\times$ less than Backstop, and 11.7k tokens/s when it sees the load one second late. Backstop achieves this by granting AI during full load whenever no neural receiver runs on that GPU: it serves 18.8k tokens/s in the full-load phases, while the share that follows the load drops to its 10% share there and serves 2.4-2.7k tokens/s. Layer-1 misses in the full-load phases are 0.03-0.04% of the TBs for every policy.

With 32 cells and a load that alternates between half and a quarter, the order is the same. Backstop serves 26.4k tokens/s and keeps 99.4% of the recoveries. The fixed 10% share serves 5.3k tokens/s at 99.4%, the fixed 30% share 22.0k at 97.7%, and the share that follows the load 22.7k at 99.0% with shares of 10 and 50%.

4.3 AI served at a steady full load

Table 4 shows the steady full load. With mixed channels, Backstop serves 22.7k tokens/s and keeps 97.9% of the recoveries. The fixed 10% share keeps 98.3% at 5.0k tokens/s, and the fixed 30% share serves 20.9k tokens/s and keeps 95.1%. With the low-correlation channel alone, every failed TB goes to the neural receiver and the neural receiver runs for 25% of the slots. Backstop serves 12.6k tokens/s at 98.2%; a fixed share must drop to 10% to keep 97.5%, where it serves 4.4k tokens/s. At the same AI served, Backstop keeps 2-3 percentage points more recoveries than the fixed shares. The high-correlation channel gives the same picture: Backstop serves 11.9k tokens/s at 99.5%, and the fixed 10% share 4.2k at 99.3%. The fixed 50% share misses the layer-1 deadline for 0.12-0.29% of the TBs, against 0.04-0.06% for Backstop.

4.4 Choosing the TBs to recover

On the urban micro channel, sending every failed TB to the neural receiver takes 4,891-4,926 runs and recovers 663-757 TBs without AI. Sending only the TBs of slots with at most nine failed code blocks takes 651-777 runs and recovers 589-746 TBs: 85% fewer runs for 5% fewer recoveries. With AI, Backstop serves 6.4k tokens/s without the rule and 33.6k with it. On the low-correlation channel with eight two-user cells, the neural receivers cannot take every failed TB. Sending only slots with at most three failed code blocks removes 40% of the runs and 22% of the recoveries, raises the recoveries per run from 0.77 to 0.99, and raises the AI that Backstop serves from 2.2k to 12.4k tokens/s.

4.5 Where recoveries are lost

We classify every recovery that a policy loses against the run without AI. With the larger model and a fixed 50%, 70%, or 100% share, 10.8%, 12.8%, and 18.6% of the recoveries are lost because no neural receiver is free before the latest start time, and 4.9%, 13.9%, and 39.1% because the neural receiver finishes after the recovery deadline. Backstop loses 3.4% to busy neural receivers and none to late results. With a 6.5 ms recovery deadline and the real-time model, the losses of the fixed shares are a conventional result that arrives after the latest start time (5-25%), because AI delays the conventional receiver by 0.2-0.4 ms, and a busy neural receiver (5-13%). Late results stay below 4%.

4.6 Where a fixed share is enough

Backstop's advantage needs a neural receiver load that fills a large part of the GPU. With the real-time model (2.7 ms per recovery) and the 11.5 ms recovery deadline, a fixed 70% share loses 1.5% of the recoveries and serves 43.1k tokens/s, against 34.7k tokens/s at 99.8% for Backstop. With the urban micro channel and the failed-code-block rule, 5% of the slots need the neural receiver, and a fixed 50% share serves 46.0k tokens/s against 33.6k for Backstop at the same 99.8%. At a steady half load, a fixed 30% share keeps 99.3-99.5% of the recoveries and serves 21.9-24.6k tokens/s, against 29.2k at 99.9% for Backstop. A fixed share that fits one load does not fit another: the share that keeps the recoveries at full load is 10%.

5 Related Work

GPU sharing in vRAN and AI-RAN. YinYangRAN [18] multiplexes LDPC decoding with other workloads by MPS shares that a learned predictor changes at intervals of at least one second. CloudRIC [19] pools accelerators for LDPC decoding, and Concordia [8] reserves capacity for vRAN tasks. AI-RAN studies [3, 12, 20, 21] place AI and RAN workloads across servers. Their radio work has one deadline and no second receiver. Backstop adds a recovery path with its own deadline and grants AI against both.

GPU scheduling. DARIS, REEF, XSched, and nvtaskset [4, 5, 9, 22] schedule GPU stages, kernels, or contexts by priority. They have no notion of a TB that may still need a neural receiver. Backstop decides from the state of each TB.

Neural receivers. DeepRx [10] and the NVlabs receivers [6, 23] design and train neural receivers, ARCHES [17] switches between an AI channel estimator and a conventional one, and OCUDU [15] exposes timing classes in the distributed unit. These works report receiver gains against their own baselines. We compare at equal LDPC effort with a production receiver and study the neural receiver as a workload that shares the GPU with AI.

6 Limitations

The channels are the synthetic channels of the public NVlabs configuration. The larger model allows four to five cells per GPU at full load; 32 cells run when half of the cells are idle. The recovery deadline follows from the TDD pattern and the PUSCH preparation time [2] and is not validated with a layer-2 stack; an 11.5 ms deadline delays the retransmission of an unrecovered TB by 5 ms. The share that follows the load is our implementation with two shares and no load predictor. Layer-1 misses are 0.02-0.06% of the TBs for Backstop and for the run without AI, which reflects the Python prototype and not a bound.

7 Conclusion

A neural receiver that decodes more TBs than a production receiver at equal LDPC effort takes twice the uplink period. Backstop runs it only for the TBs the conventional receiver fails and that it is likely to decode, under a recovery deadline, and grants AI inference in pieces that keep the layer-1 deadline and the recovery deadline. Under a changing load with mixed channels, Backstop serves 27.4k tokens/s while keeping 98.6% of the recoveries, $5.0\times$ a fixed GPU share and $1.17\times$ a share that follows the load.

References

- [1] 3GPP. 2020. *Study on Channel Model for Frequencies from 0.5 to 100 GHz*. 3rd Generation Partnership Project. <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3173> Technical Report 38.901.
- [2] 3GPP. 2024. *NR; Physical Layer Procedures for Data*. 3rd Generation Partnership Project. <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3216> Technical Specification 38.214.
- [3] Ganesh Ananthanarayanan, Matthew Balkwill, Xenofon Foukas, Zhihua Lai, Bozidar Radunovic, Connor Settle, and Yongguang Zhang. 2025. Distributed AI Platform for the 6G RAN. In *ACM Workshop on Open and AI RAN*. ACM, New York, NY, USA, 15–21. <https://doi.org/10.1145/3737900.3770167>
- [4] Amir Fakhim Babaei and Thidapat Chantem. 2025. DARIS: An Oversubscribed Spatio-Temporal Scheduler for Real-Time DNN Inference on GPUs. In *ACM/IEEE Design Automation Conference*. IEEE, Piscataway, NJ, USA, 1–7. <https://doi.org/10.1109/DAC63849.2025.11132423>
- [5] Joshua Bakita and James H. Anderson. 2025. Hardware Compute Partitioning on NVIDIA GPUs for Composable Systems. In *ECRTS*, Vol. 335. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Wadern, Germany, 21:1–21:25. <https://doi.org/10.4230/LIPICs.ECRTS.2025.21>
- [6] Sebastian Cammerer, Fayçal Ait Aoudia, Jakob Hoydis, Andreas Oeldemann, Andreas Roessler, Timo Mayer, and Alexander Keller. 2023. A Neural Receiver for 5G NR Multi-User MIMO. In *IEEE Globecom Workshops*. IEEE, Piscataway, NJ, USA, 329–334.
- [7] Sebastian Cammerer, Reinhard Wiesmayr, Fayçal Ait Aoudia, Jakob Hoydis, Tommi Koivisto, Jakub Zakrzewski, Ruqing Xu, Pawel Morkisz, Chris Dick, and Alexander Keller. 2024. Real-time 5G NR Multi-user MIMO Receivers. <https://github.com/NVlabs/neural-rx>.
- [8] Xenofon Foukas and Bozidar Radunovic. 2021. Concordia: Teaching the 5G vRAN to Share Compute. In *ACM SIGCOMM*. ACM, New York, NY, USA, 580–596. <https://doi.org/10.1145/3452296.3472894>
- [9] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. 2022. Microsecond-Scale Preemption for Concurrent GPU-Accelerated DNN Inferences. In *USENIX OSDI*. USENIX Association, Boston, MA, USA, 539–558. <https://www.usenix.org/conference/osdi22/presentation/han>
- [10] Mikko Honkala, Dani Korpi, and Janne M. J. Huttunen. 2021. DeepRx: Fully Convolutional Deep Learning Receiver. *IEEE Transactions on Wireless Communications* 20, 6 (2021), 3925–3940. <https://doi.org/10.1109/TWC.2021.3054520>
- [11] Jakob Hoydis, Sebastian Cammerer, Fayçal Ait Aoudia, Avinash Vem, Nikolaus Binder, Guillermo Marcus, and Alexander Keller. 2022. Sionna: An Open-Source Library for Next-Generation Physical Layer Research. arXiv:2203.11854 <https://arxiv.org/abs/2203.11854>
- [12] Haiyuan Li, Yulei Wu, and Dimitra Simeonidou. 2026. Deadline-Driven Hierarchical Agentic Resource Sharing for AI Services and RAN Functions in AI-RAN. arXiv:2605.07547 [cs.NI] <https://arxiv.org/abs/2605.07547>
- [13] NVIDIA Corporation. 2026. *Aerial CUDA-Accelerated RAN Documentation*. NVIDIA. <https://docs.nvidia.com/aerial/cuda-accelerated-ran/>
- [14] NVIDIA Corporation. 2026. *Multi-Process Service Documentation*. NVIDIA. <https://docs.nvidia.com/deploy/mps/>

- [15] Timothy O'Shea, Matthew Pennybacker, and Andriy Kharchenko. 2026. The OCUDU dApp Platform: An Open Runtime and E3 Interface for Real-Time AI-RAN. arXiv:2609.07843 [cs.NI] <https://arxiv.org/abs/2609.07843>
- [16] Qwen Team. 2024. Qwen2.5 Technical Report. arXiv:2412.15115 <https://arxiv.org/abs/2412.15115>
- [17] Neagin Neasamoni Santhi, Davide Villa, Michele Polese, Salvatore D'Oro, Yunseong Lee, Koichiro Furueda, and Tommaso Melodia. 2026. ARCHES: Adaptive Real-Time Switching of AI Models for the RAN. arXiv:2604.23397 [cs.NI] <https://arxiv.org/abs/2604.23397>
- [18] Leonardo Lo Schiavo, Jose A. Ayala-Romero, Andres Garcia-Saavedra, Marco Fiore, and Xavier Costa-Perez. 2024. YinYangRAN: Resource Multiplexing in GPU-Accelerated Virtualized RANs. In *IEEE INFOCOM*. IEEE, Piscataway, NJ, USA, 721–730. <https://doi.org/10.1109/INFOCOM52122.2024.10621380>
- [19] Leonardo Lo Schiavo, Gines Garcia-Aviles, Andres Garcia-Saavedra, Marco Gramaglia, Marco Fiore, Albert Banchs, and Xavier Costa-Perez. 2024. CloudRIC: Open Radio Access Network (O-RAN) Virtualization with Shared Heterogeneous Computing. In *ACM MobiCom*. ACM, New York, NY, USA, 558–572. <https://doi.org/10.1145/3636534.3649381>
- [20] Syed Danial Ali Shah, Maryam Hafeez, Abdelaziz Salama, and Syed Ali Raza Zaidi. 2025. Proactive AI-and-RAN Workload Orchestration in O-RAN Architectures for 6G Networks. arXiv:2507.09124 [cs.NI] <https://arxiv.org/abs/2507.09124>
- [21] Syed Danial Ali Shah, Zeinab Nezami, Maryam Hafeez, and Syed Ali Raza Zaidi. 2025. The Interplay of AI-and-RAN: Dynamic Resource Allocation for Converged 6G Platform. In *IEEE INFOCOM Workshops*. IEEE, Piscataway, NJ, USA, 1–6. <https://doi.org/10.1109/INFOCOMWKSHPS65812.2025.11152939>
- [22] Weihang Shen, Mingcong Han, Jialong Liu, Rong Chen, and Haibo Chen. 2025. XSched: Preemptive Scheduling for Diverse XPU's. In *USENIX OSDI*. USENIX Association, Boston, MA, USA, 671–692. <https://www.usenix.org/conference/osdi25/presentation/shen-weihang>
- [23] Reinhard Wiesmayr, Sebastian Cammerer, Fayçal Ait Aoudia, Jakob Hoydis, Jakub Zakrzewski, and Alexander Keller. 2024. Design of a Standard-Compliant Real-Time Neural Receiver for 5G NR. arXiv:2409.02912 <https://arxiv.org/abs/2409.02912>